Unity and Wwise Contribution Guide

This documentation gives an overview and some technical details on how to contribute new **city models/maps**, **individual 3D objects**, and **sound sources**. All resources must be prepared in the following ways for integration into City Ditty by Richard – there is no interface for doing so yourself. It assumes some pre-requisite knowledge on these topics and is not a guide on how to use Unity or Wwise.

Disclaimer: I try to support as many things as possible but am sometimes limited by knowledge and time to support everything. Please contact me ahead of time to discuss. I am a single researcher that programs everything, in addition to my other responsibilities. See the **FAQ** at the bottom of this document for commonly asked questions.

Cite this work: If you want to cite the project or software itself, please cite one of its most recent publications. i.e.,

> Yanaky, R., Tyler, D., & Guastavino, C. (2023). City Ditty: An Immersive Soundscape Sketchpad for Professionals of the Built Environment. *Applied Sciences*, 13(3), 1611. https://doi.org/10.3390/app13031611

If you need to cite information specifically from this guide (e.g., technical recommendations), you can cite it as follows:

> Yanaky, R. (2025). City Ditty User Guide [Unpublished user guide]. https://www.sounds-in-the-city.org/en/projects/project23

Contents

Unity and Wwise Contribution Guide 1

What can you create and contribute? An overview 2

IMPORTANT! Compatible versions, rendering pipelines, and assets 3

Desktop vs Head-Mounted Device (HMD) versions 3

Unity Asset Store & Render pipeline compatibility 4

Scale of city space and objects 5

3D Environment / City Map 5

The Unity Hierarchy 5

Ground Layer 6

StreetLight Tag (Automatic lights at night) 7

Sound obstruction on walls and other objects 7

Sound reflections 7

Traffic/cars....8
User-Interactable Objects....8
Required Audio/Visual Settings per interactable object....8
Size and Form of User-Interactable Objects....8
Trees – both user interactable and non-interactable....9
People....10
Exporting Unity Assets....11
Exporting 3D Environments (Maps)....11
Exporting Interactable Objects (Prefabs)....12
Sound Sources / Audio files....12
Optimising audio files for real-time virtual environments....12
Recording sound sources (or finding them online)....14
Adding sound through Wwise....15
Audio Hierarchy Structure....15
Audio Source settings....17
Wwise Events Hierarchy Structure....20
Changing existing SFX or Random Containers....22
Frequently Asked Questions (FAQ)....22

What can you create and contribute? An overview

Currently, the best way for collaboration is to create the following items individually and send them to me to integrate into unity. Wwise is much simpler. I can send a working Wwise project for you to work with directly. After I integrate these, I can send back a working program using your assets. Contact me ahead of time for timelines.

Basically, you need to create anything from 1-3 below, then send them to me:

1) **A 3D environment that works well in Unity.**
    1. This 3D environment is not user-interactable.
        - Other specialized software already exists to create and modify the 3D environment.
        - Citty Ditty focuses on manipulating sounds and other contextual factors that affect sound, like time of day.
        - **The format of this is a Unity Scene.**

2) **Individual 3D objects that can be interacted with by the user.**

1. These also act as potential sound sources.
2. You can also create non-sound sources such as sound barriers and hills that you can place on top of the existing 3D environment to affect smaller changes.
3. **The format of these are Unity Prefabs.**

3) Sound clips that are emitted from sound sources.

1. Sound is rendered in Wwise.
2. Background sound (ambient room tone) should be an Ambisonic recording.
3. All other sound sources are all ***point sources***, meaning the sound will be emitted from a single point, even if your object is larger (e.g. a large fountain). There are methods in Wwise to take this into account.
4. All sound sources need to accompany a 3D model from (2), as all sound sources need something tangible to interact with.
5. **There is a Wwise Template with instructions to follow, below.**

NOTE: Illegally obtained assets are not permitted.

1. By submitting any assets, you are stating that you have the rights to use these resources, either through open-licenses, content created yourself or purchased/licensed from another source.
2. If anything has been purchased or licensed (copyrighted material from elsewhere, basically), please include an excel sheet listing these resources and where they came from. I need this information to know what I can and cannot release to others.
3. If you "found" an asset on the internet, you need to state where you found and ensure that you have the rights to use it. *Don't use websites that promote piracy.*
    - If in doubt, contact the creator and ask for written permission in a simple email. If you cannot contact the creator or find information regarding its rights, do not use it.

IMPORTANT! Compatible versions, rendering pipelines, and assets

Currently, City Ditty is running on the following versions of Unity and Wwise. These will be updated at a later point, but is otherwise not possible to do right now. Please make sure your assets work with these versions.

- Unity 2021.3.0f1
    - Built-in Rendering Pipeline Only.
- Wwise 2022.1.1.8100

Desktop vs Head-Mounted Device (HMD) versions

The core functionalities of City were programmed in the desktop version, then was eventually split off into two separate versions: The desktop version and the Head-Mounted Device (HMD) versions, as they require drastically different interactional components, as well as different virtual rendering settings. While I try to keep them both updated somewhat equally, it is possible that there is a bug

in one version that is not present in the other. If reporting issues, please include which version you are using.

Limitations of HMD version

Overall, the HMD version is the same as the desktop version in terms of functionalities. However, due to hardware and various renderer limitations, a few things are different.

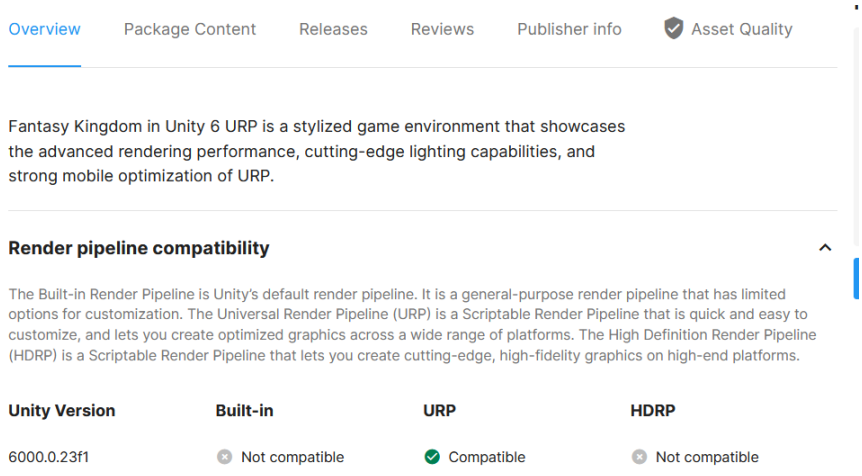
- HMDs do not render objects in the distance as far as the desktop version is capable of. This is an inherent limitation that I cannot program around. If looking far into the distance, eventually things will get cut off. As HMDs and Unity increases in capabilities, this should improve in the future.
- Clouds do not render properly in HMDs with the current weather system. As such, I have disabled tas ability to include them. They remain available in the desktop version.

All audio-renderings are identical between the two versions.

- These are the only two limitations that I am currently aware of. Please consider these if doing comparative desktop vs. HMD studies.

Unity Asset Store & Render pipeline compatibility

The unity asset store can provide a lot of 3D objects but be very careful on what you buy ***(and make note of them in your excel document of copyrighted material)***. All 3D objects need to run with the Built-in render pipeline and be compatible with the current version of Unity. Most assets specify what will work and what does not work. For example, this asset below is only compatible with the Universal Render Pipeline and thus, would ***not*** work with this version of the software. I don't know too much about the rendering pipelines, but as we are not creating AAA game visuals, it should not impact us very much. We are audio-focused. Still. Do not waste any money purchasing something that you cannot use. If you somehow have the knowledge to convert assets to work in the Built-in render pipeline, you are free to do so. I am unable to do so.



Scale of city space and objects

1 Unity size unit is equal to 1 meter. This is a Unity standard that needs to be followed.

Keeping close to this 1:1 ration is important because:

1. 3D objects should be roughly equal in scale to each other, otherwise your user will find it unusual and distracting. This could influence their experience in the space you developed.
2. Sound propagation relies on an equation that treats these as 1:1. If you move too far outside of this ratio, the propagation system will be off.

That said, the exact precision of your sizes is up you to. 3D world building is very labour intensive and if something is off by a little bit, it likely doesn't matter too much, holistically. However, if the object is a central focus for designs, more efforts should be made for accuracy.

3D Environment / City Map

City Ditty is designed for sound manipulation, not physical environment manipulation, such as topography, for example. When constructing a 3D environment, it is important to decide what you do want/need to be interactable as a sound source, and what will not. As such, if you are creating your own 3D environment of a city space, park, etc., it should already contain everything that you do not intend on letting the user manipulate. The following section on Interactable Prefabs gives more detail on those.

The Unity Hierarchy

City Ditty acts as an interaction layer for sound, placed overtop of a 3D environment. This 3D environment must then utilize several layers and tags to hook it into the interactive components of City Ditty. To support this, City Ditty requires the unity hierarchy to be structured in a particular way to help make this connection:

1. The 3D environment must be created and saved as a ***single Unity Scene.***
    - Name the scene after the location you are modeling.
2. ***The city model itself should also be saved as a single unity object***, within the scene.
    - This allows the entire environment to be moved around as a single unit, if necessary.
    - Use a 3 character model code for your city.
3. Similarly themed objects should all be stored under a single parent objects, within the city model.
    - E.g., Light sources, road blocks, ground, trees, etc. each get their own parent object.
4. Different layers and tags must be applied to these objects to connect them to the functions of City Ditty.
    - E.g., Ground and lights, as explained below.

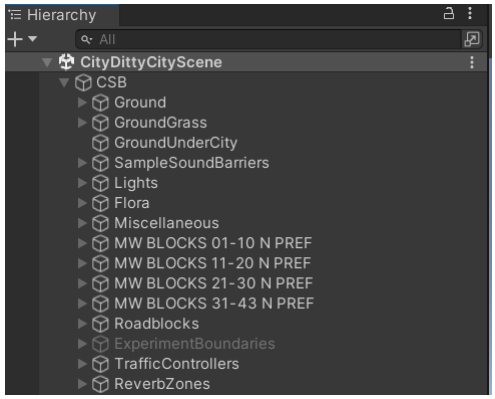


Figure 1 - All similarly themed objects should be stacked together in single parent objects. The entire 3D space should be located in a single parent object (in this case, CSB).

This organizational system helps me find issues that may need fixing quickly, as well as apply different functionalities to different sets of objects. For example, the **ExperimentBoundaries** gameobject contains all barriers necessary for experiment mode in this map, which can be enabled or disabled, according to user choice.

while the "StreetLight" tag allows for automated nighttime illumination. The "Soundsource" tag indicates an object that can be interacted with (i.e., moved around or sound settings changed). Any user placeable object (e.g. birds and fountains) should be saved as a unity prefab with no external dependencies to allow re-use in design.

Ground Layer

Ground is any walkable space, such as roads, sidewalks, etc. All objects in Ground should have 'Ground' applied to the Layer of all of the ground objects. By applying the Ground layer, you also enable sound obstruction on these surfaces. It does not include sound reflections, as of yet.

If this layer does not exist in the dropdown, you can add it via the 'Add Layer' option in the dropdown of the inspector window.

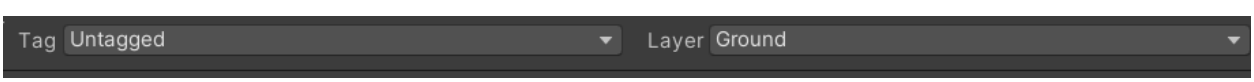


Figure 2 - Setting Layers on objects.

A Unity Collider component must also be attached to any object that you want to walk on, block/reflect sound, or otherwise interacted with; otherwise they remain visual props, only. By adding layers and tags to different objects, it signals to City Ditty what their function is. The "Ground" layer denotes a solid surface that will block sound and can be walked on.

All walkable surfaces need both the Ground layer + unity collider component attached to them. If they are not both there, the user will fall through the ground.

StreetLight Tag (Automatic lights at night)

Streetlights are automatically turned on at night when the sun sets and turn off again when the sun rises. This corresponds to the sun position and not the time of day. If you want to include any streetlights that follow this behaviour, you need to apply a "StreetLight**" tag** to the game object that has the <Light> component attached to it. Any lighting settings beyond this are up to you. I am not a lighting expert and use mostly default settings, other than Type, Range, Color, and Intensity.

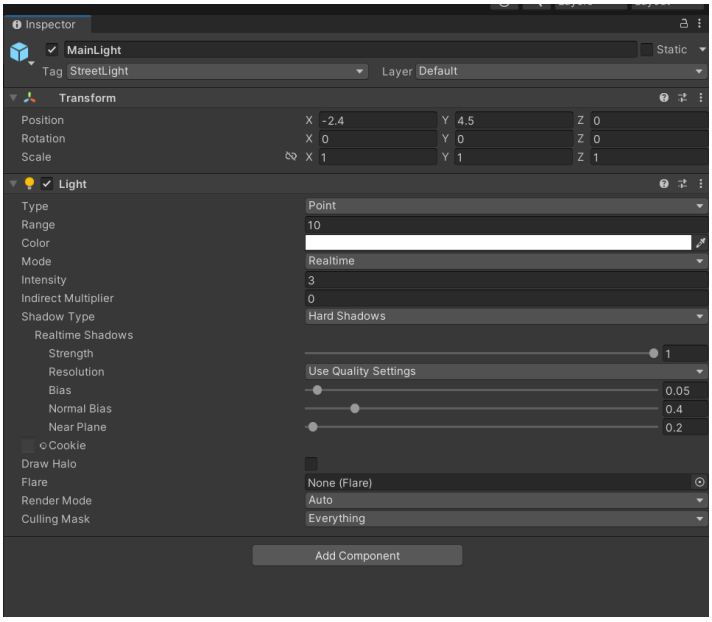


Figure 3 - Setting Tags on objects.

Sound obstruction on walls and other objects

Currently, the Ground layer is the only way to apply sound obstruction to objects. This also means that I also apply the Ground layer to city buildings and other noise barriers as well. In a later version there may be different ways to have different material absorption and reflections, but that has not been implemented for this version.

Sound reflections

There are currently no sound reflections. Given that we are always designing large outdoor areas, this has not led to issues as of yet. We do, however, have the ability to place additional reverberation over large spaces to help simulate this to a small degree. For example, our city park has little to no reverb due to it being a large open space. However, we added a small amount of artificial reverberation to sidestreets as they are surrounded by buildings, which would cause more reflections.

This was originally setup by someone more familiar with acoustics. Instructions for including these will be included in a later version once it gets expanded upon.

Traffic/cars

Vehicles currently use a very simple system which can be expanded upon in the future, as projects allow/need.

Currently vehicles only go in a straight line from a start to finish point. Cars will spawn automatically as long as traffic is enabled in City Ditty. A multiplier is used to increase traffic during different times of the day and decrease it at night. This currently has no interface to change the flow of traffic at runtime; it is either on or off. In the editor while preparing it, however, you can change how often they spawn and how fast they go. Given the amount of vehicles required to make a realistic space, object pooling is used in conjunction with animators to move the vehicles. If you choose to modify this, ***DO NOT USE PHYSICS-BASED MOVEMENT***. This will be very resource intensive and is unnecessary.

Engine sounds are attached to each vehicle, and you can specify a randomized chance of having a vehicle honk its horn during it's drive, as well as a chance for a car to be playing loud music when spawned. A future change is planned that will allow users to change the speed of traffic, but we have no ETA yet.

Cars are not directly user-interactable. They can only be turned on and off, at present. As such, they are considered to be part of the 3D Environment, rather than a User-Interactable Object.

User-Interactable Objects

User-interactable objects are anything that the user can directly add, remove, move, or change audio playback settings on, such as birdhouses, fountains, people, picnic tables, trees, sound barriers, etc. These must all be saved as a unity prefab with no external dependencies to allow re-use in design.

Required Audio/Visual Settings per interactable object

All user-interactable objects require:

1. A short name that is viewable by the user.
2. Specify if a silent object or include the affiliated sound recording(s).
3. Active Hours, if not silent.
    a. Both Simple (in 6-hour blocks) and Advanced (by the hour)
    b. Specify which of these modes to enable by default.
4. Visibility status: Always visible or disappears outside of active hours.

Size and Form of User-Interactable Objects

All sound sources are attached to objects. If they are user-interactable, make sure that they are both large enough to be interacted with, and make sense to the user.

For example, for the purposes of biophilic design, you may wish to include different insects, bird, etc. Some of these may be very small and hard for the user to grab or interact with. Likewise, a user/soundscape designer could rightfully think "it's unrealistic to place a bird, insect, or other living creature here as I have no control over nature". This is why City Ditty uses birdhouses, for example, to represent birds. We can teach designers different ways to naturally attract wildlife using such types of techniques, while giving a slightly larger object that is easier to interact with. With any type of living creature you could ask similar questions: Are insects attracted to certain types of bushes or trees? These could be used as the sound source origin to make things more realistic, as well as informative to the designer.

Ultimately, however, this is up to the purpose you have. While I personally like the concept of using bird houses to attract the sounds of birds, this is a decision that benefits the designer at design time. If someone is just walking around and evaluating the virtual soundscape, having a bird or cloud of insects may seem more realistic.

On a technical point, for the creation of 3D assets (e.g., birds, people), designers can employ any 3D modeling software of their choice (e.g., Rhinoceros 3D, Grasshopper, Blender or Maya), it is essential to follow standardized export protocols to ensure interoperability into Unity, including maintaining a consistent upright and forward orientation in their software of origin, as orientations may be changed during software runtime. It is recommended to use formats such as FBX or OBJ, paying special attention to maintain aforementioned compatibility requirements (version of Unity, rendering pipeline, and geometric scale of 1:1). Textures should be delivered in PNG or JPG format, with maximum resolutions of 2048x2048 for large surfaces and 512x512 for smaller elements and contain a relatively low polycount. Components using physics simulations should be omitted unless strictly necessary. Consider using animations, instead. Such optimizations are required as user designs can quickly become complex with many objects and sounds in their designs, causing large influxes of processing power required when poorly optimized. There is no limiter in place to curb complex designs.

Trees – both user interactable and non-interactable

The creation of trees is a large topic of practice when creating 3D environments. Since there are so many trees outside, it is better to have more variation in how the trees look. However, this conflicts with the notion of prefabs, as prefabs are always the same, or have simple variants, to support ease of re-use. This has resulted in two types of tree assets being created: Simple trees that can be turned into unity prefabs, and procedurally generated trees that have a lot more variation but cannot be turned into prefabs.

Since the City Ditty City is mostly urban streets, it just uses three very simple tree models with a slight animation of blowing in the wind. These are all prefabs that can be placed directly in the 3D environment scene (not user interactable), as well as be used as user-interactable objects. I also attach simple audio clips of leaves rustling in the wind to them.

The Parque O'Higgins model, created by the Chucao lab (University of Chile) on the other hand, focuses on a large park space that includes many trees. As such, these trees use a procedurally

generated method to increase variation of the trees. As a result, it is much easier to create a large tree-covered area with more variation in tree appearances. However, they cannot be selected individually. This means they cannot inherently be treated as sound sources (e.g. for sounds of leaves blowing in the wind). Also, any tags or layers must be applied to the entire group, rather individually. Furthermore, you cannot (typically) export them as individual objects/prefabs, as part of the procedurally generated data is stored as part of the Unity Scene, and not just the object, itself.

In short: Trees that can be made into prefabs can be used as user-interactable objects. All others can only be part of the immutable 3D environment/map. Consider the interactional goals of your 3D environment before deciding on which type(s) to use.

People

I am using this asset for people, along with a collection of different animations that I apply to them.

https://assetstore.unity.com/packages/3d/characters/humanoids/humans/toon-city-people-163720

Currently, people only stand in stationary spots; I have no system for path finding or walking down sidewalks as of yet. I have plans to add this, but it is of lower priority for now. You are free to create and use different people or use the existing ones.

Currently, I have a collection of English and French voices. I highly recommend finding or recording short conversations in your local dialect. If you use a lot of foreign languages that are not commonly used, this will sound and feel unusual to your users. Also, collect several different recordings for both men and women. Since you may have many people talking in a space, you don't want to have them all saying the same thing. You can randomize what they say from. The bigger your list of voice clips, the better because there will be less repetition.

Recall that the sound sources come out of a single point in space. These points are not yet customizable. Currently, they come out of the 'anchor' position in Unity, which is typically the center of the object that they're attached to. In the future, these will be customizable to have voices, for example, come out of a person's mouth. Similarly, if you have a recording of a conversation between multiple people, it will come out of the same point. Make sure to include multiple people in the design to avoid an audio-visual disconnect.

When listened to from a distance, or even on a desktop computer, it is highly unlikely that users would be able to tell the exact point in which the sound is coming from. E.g., Whether the voice coming from their belly or their lips. However, in head-mounted virtual reality, with the enhanced immersion, people may be able to pinpoint these sources more accurately if they walk right up to a group of people. While this source precision will be enhanced in the future, it is more than sufficient to give an overall impression of a soundscape, which is our current goal.

Exporting Unity Assets

The following instructions are provided so that you can export your work into a format that I can integrate into my project. Basically, they need to be self-contained Unity Scenes and/or Prefabs that I can use, without it being part of your Unity project. This is followed by some troubleshooting tips that we ran into in an earlier collaboration, which are particularly useful if you used generative tools to create your 3D environment.

Exporting 3D Environments (Maps)

Give your Unity Scene a short descriptive name. Then, in the project window, locate and right click on your scene. Select ***Export package***, and export! If you used any generative tools, there may be extra steps though.

To check if it worked properly, create a new project and import your scene. Do not reinclude any packages that included the generative tools that you used during design time – these may reintroduce external dependencies.

1. Run your project in the Unity editor. Does it look OK? If not, you didn't include something in your export.
2. Create a test build and check for errors. Some issues are overlooked in the editor, but need to be resolved at build time. File → Build settings → Clean Build.

If everything works, it should be fine! Send it my way!

Trouble shooting tips:

1. If you used all self-contained assets, I don't expect there should be any problems. You can check if it's self-contained by trying to create a prefab of your entire city. Since the city should all be saved under a single parent game object, you can simply drag this game object from the Hierarchy window into the Project window. This will prompt the creation of a Prefab.
    a. If it produces an error, check the error and try to resolve it. It could be something simple like "missing script", which can be removed from a GameObject.
    b. If the missing script was important, that may mean that the asset was not self-contained to the GameObject. It may be saving part of the data in the Unity Scene.
    c. If this doesn't work, that's OK! We don't need a Prefab of the entire city. It's just a test. This just means that some of the data itself is likely stored in the Unity Scene.
2. If any generative tool is used to create the 3D environment, there is a high likelihood that some of the data will be saved in the Unity Scene. That's fine! It may just limit a few potential interactions.
    a. When a generative tool is used, sometimes it leaves scripts on the individual GameObjects that it produces. These scripts are likely what help manipulate its form in the editor. These might not be exported, resulting in "Missing Script" components on the GameObjects.

b. If they still render properly in the editor after opening them in a new project, you're probably fine! But they will need to be removed, as Unity will not compile a project with missing scripts.
c. To do this, try creating a build of your project. This will produce several errors including the names of the objects that are missing the scripts.
d. You can search for these scripts by name in the Hierarchy.
e. Remember you can select multiple objects in the hierarchy and then remove the missing script component from the Inspector window of them all at once.
    i. If the missing script component is not visible after selecting multiple objects, it means that you selected an object without the missing script. Components only appear in the Inspector if they are shared by all objects.

Exporting Interactable Objects (Prefabs)

This one is easy. Import all of your intended interactable objects into Unity. Then transform them into prefabs, if they are not prefabs already. The easiest way to do so is by dragging the GameObject from the hierarchy to the Project window.

Create a new folder in your Project ->Assets folder with a descriptive name of your project. E.g., 'ChucauLabPrefabsDate'. Drop all of your Prefabs in there. Right click your new folder, select Export, and done!

Prefabs must be self-contained, so there shouldn't be any issues. However, it's always a good idea (and time saver) to create a new Unity project and try importing them there. This can quickly identify any issues to avoid any extra back and forths.

Sound Sources / Audio files

Two types of audio files are used.

1. Ambisonic recordings for background / room tone or other important sources.
2. Mono or Stereo recordings for individual sound sources.

Optimising audio files for real-time virtual environments

Optimizing the number of audio channels is crucial in real-time applications, as each channel can drastically increase processing demands and memory usage. For an application like this where there are a ton of user-definable sound sources, this is very important. You are highly encouraged to use single-channel/mono recordings for most sound sources. Wwise will work its magic to spatialize it and make it sound realistic. In Wwise, you can further specify other details on how it's rendered, like spread, propagation, etc. If it sounds good enough with a single channel, that is best. If there is a particular sound source that you need to really emphasize the spatial qualities of, extra channels can help provide a fuller spatial depth.

Also keep in mind the frequency that users will want to use the sound source. Cars, for example, are plentiful, with many on the road at a time. Given this, short, simple, single-channel audio clips should be used for them.

Background sound / room tone

Currently, we only use 4-channel ambisonic recordings for background sound that is always centered on the user. These are recordings taken at 4am to get a general background sound. This gives a rich background sound that's consistent when you turn your head. As it is always centered on the user and always playing, it is a good candidate for multiple channels.

When recording these, it is OK to have non-disruptive sounds in the far distance (e.g. sound of the occasional car) but there should be no sounds coming from up close. As the background recording will always be played on loop, this will be noticed and become irritating – especially if it's close enough to see, but nothing is there.

Ideally, you would have a background recording late at night, and one during the day. Once the sun comes up (and the lights turn off), this background sound is switched to a louder daytime background recording.

However, it can be very hard to get a decent recording during the day, as there would no doubt be many 'close' sound sources that will be disruptive. What we did was use the nighttime recording in the software at night, then use a duplicate recording during the day where we just increased the sound levels. For our purposes, it is effective, as we rely on the user-placed sound sources to create most of the soundscape.

Important!! Tips for decreasing file size

While there is no direct requirement on file size, a lot of efforts should be placed on optimizing every audio file. Always assume that your users will use a lot of them. Who knows what they will create? Very creative or ambitious users could potentially add hundreds of sound sources! I have not stress tested the software to determine upper limits, as this will always be dependent on the power of somebody's computer. Also, given the nature of audio files, they can quickly become extremely large in size and have massive impact on the software's performance. Large audio files and large 3D texture sizes are among the worst perpetrators for draining unnecessary processing power.

Here are common methods to shrink your file sizes. You should listen to each clip before and after to determine what is OK. Also consider that most sounds will be perceived in the background and, as such, do not require really high quality; a lot of degraded sound quality will not be noticed by the user because they are listening to the soundscape holistically.

1. **Adjust Bitrate:** Lowering the bitrate can significantly reduce file size without noticeably impacting audio quality. For background sounds, reducing from high bitrates (e.g., 320 kbps) to 128 kbps or even lower can be very effective.

2. **Reduce Sample Rate:** For sounds that don't require extreme fidelity (like ambient backgrounds), consider reducing the sample rate from 44.1 kHz to 22.05 kHz (or perhaps even lower!)

3. **Use Compression Formats:** Wwise only accepts a limited amount of file formats, but then will do its own conversion to compressed formats when building the soundbank. You can specify what to use. I am not an expert on this topic, but here were a few examples. There may be newer formats or recommendations:
    a. **Vorbis:** Great for general audio compression, particularly music and ambient sounds. It provides a good balance between quality and file size.
    b. **ADPCM**: A lower-complexity format, suitable for short sounds like footsteps or UI interactions, as it provides moderate compression without significant CPU load.
    c. **Opus:** An efficient codec often used for streaming audio or spatial sounds, offering high-quality compression at low bitrates.

4. **Convert to Mono:** As stated above, mono is best for most sound sources such as voices, background sounds, or any sound without too much spatial detail. Reserve multi-channel formats for critical spatial audio effects. Use these sparingly. Some objects like fountains or HVAC systems may potentially benefit from having additional channels to help with the spread.

5. **Trim Silence and Crop Audio:** Remove any unnecessary silence at the beginning or end of files. Trimming audio clips down to just the needed length can save considerable space.

6. **Loop Shorter Clips:** For repetitive background sounds, create short seamless loops instead of longer recordings. This can maintain audio continuity without large file sizes. If it is hard to make a seamless loop, you could low the sound levels a bit at the start and end of a clip (fading in and fading out).

Recording sound sources (or finding them online)

Individual sound sources should be ***isolated*** or ***dry***, meaning that there should be little-to-no background sounds in the recording. It should also be free of reverberation. This gives us the most freedom to manipulate the sound source under different conditions. It also prevents it from sounding out of place. E.g. if there is a person standing outside in the park speaking, it would be very unusual to have reverberation in their voice. It is also disruptive if you have background noises. For example, we had a recording of leaves on a tree rustling a bit in the wind. The tree itself

sounded good, but it had feint sounds of cars in the background. This felt very unusual if there were no cars in the area. As such, it was discarded and replaced.

The highest-quality recordings can sound wonderful when they are experienced with a single playback! But due to the nature of user-created simulations (and hardware limitations), we need to lower/optimize the quality of the recordings to support design time.

Adding sound through Wwise

All audio clips must be inserted into Wwise (version 2022.1.1.8100) and be setup with appropriate sound levels, propagation, and other various settings to create a realistic and balanced soundscape. Audio sources must all come with Wwise Events that can be called from Unity in order to start, stop, mute, and unmute them. This section gives an overview of how to setup the structure of your Wwise project, along with pointing out which settings must be changed.

Disclaimer: My own spatial acoustic knowledge is less specialized, having had an acoustician and sound engineer help with the initial Wwise setup. As such, I may not be able to answer some technical questions. However, the working examples in Wwise can provide templates for adding similar sound sources. E.g., birds, fountains, HVAC, people, etc. *This set of instructions does not replace lessons on how Wwise works*. ***Wwise provides free lessons online***. The below instructions gives instructions on how to best structure things to facilitate collaboration with the developer.

Audio Hierarchy Structure

Audio files are added to Wwise in the Audio Tab of the Designer layout interface (accessed through the top Layouts tab). All audio files need to be added to a Work Unit, located in the Actor-Mixer Hierarchy folder. The figure below shows three different Work Units: 1) Default Work Unit, 2) WorkUnitFromChile, and 3) WorkUnitNewContributions. Currently, all audio sources from the original City Ditty City are stored in Default Work Unit, with additional sounds from the Chucao lab (Parque O'Higgins) saved in WorkUnitFromChile. These separate work units are necessary to help organize sound sources from each project and location. ***Any new contributions should be added to the WorkUnityNewContributions folder.*** This is important, as each Work Unit gets its own file in the windows file system that can be copied between Wwise projects. This allows me to import any additions without risk of overwriting other settings; projects cannot be directly merged, nor can I easily swap Wwise projects, as this may break existing integrations with Unity.

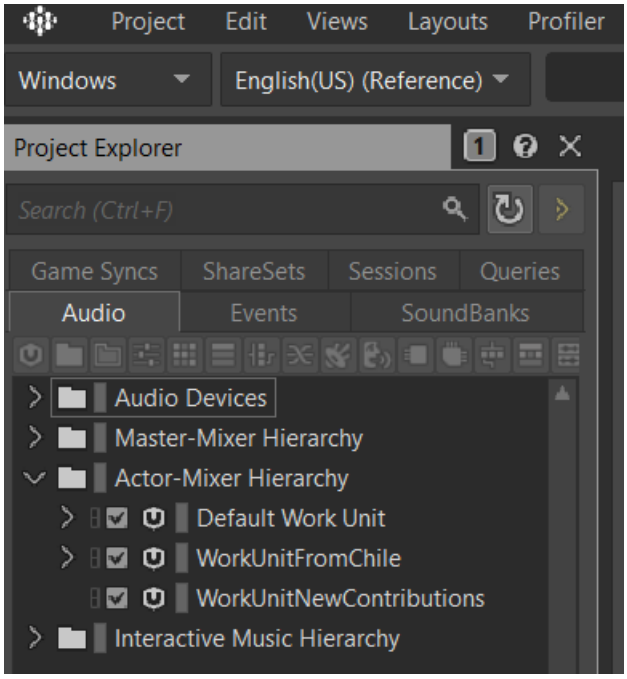


Figure 4 - Audio hierarchy structure.

The structure within the new work unit should follow thematic structures for organization, as exemplified in the following figure. The names needn't be the same, but such organization allows for easier inspection and ease of use.

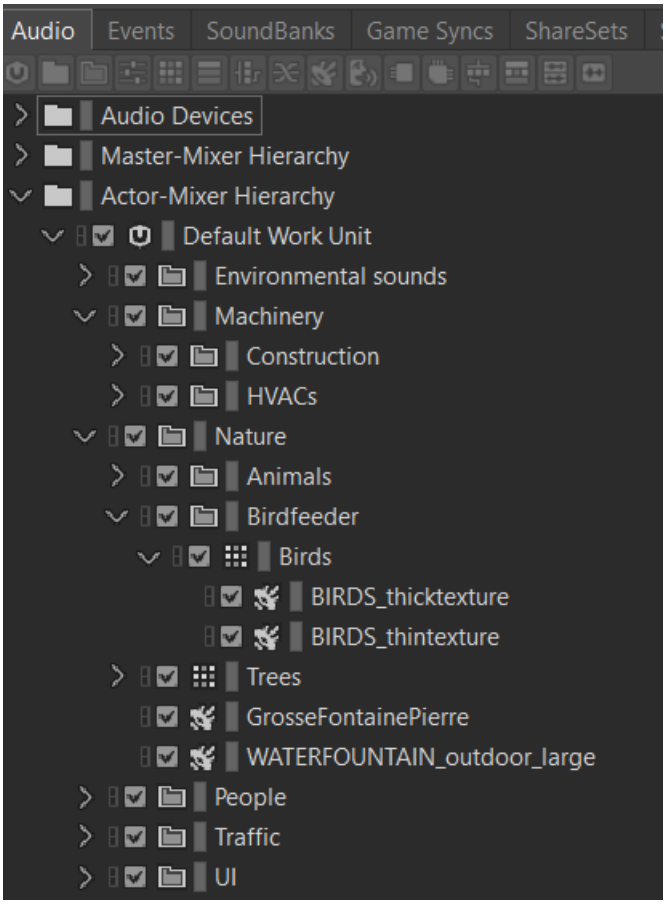


Figure 5 - Audio hierarchy with thematic organization

Audio Source settings

Audio files have many settings that must be set in order to produce proper spatialized audio. Individually, they are typically created as *SFX objects*. Ideally you will have multiple different recordings for any sound source that you expect variation from. E.g., bird songs, people talking, music. This is because as the user designs a soundscape, they will inevitably place more than one object of the same type. If all people, for example, play the same audio clip, saying the same things, it will quickly become disruptive and break the user's plausibility of it being a real space. As such, having multiple recordings can help avoid this. When you have multiple SFX objects, you must create a *Random Container* to hold them all. This random container is treated as a single sound event which can randomly select and playback a different audio clip at random (Figure 6). If the sound source is fairly consistent or would even be disruptive to suddenly change (e.g. fountain, HVAC, or car engine), a random container is not necessary – a single SFX object can be used.

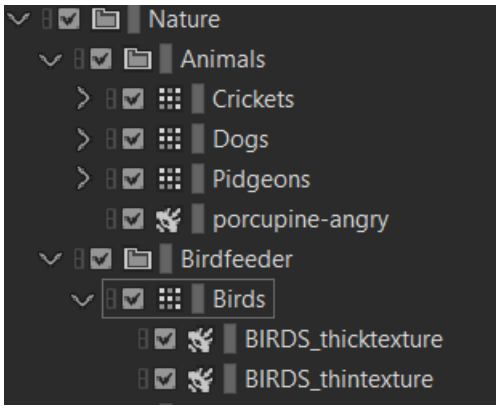


*Figure 6 - The random container, **Birds**, holds two different bird recordings. Birdfeeder is just a simple folder for organization.*

Any setting changed on the random container will apply to all audio sources contained within (Figure 7). Be sure to set Play Type to Shuffle to enable this randomization. Any further settings that are set at the random container level will apply to all child audio sources, within it. Setting the output bus, attenuation curves, etc. must also be set. Propagation curves should be taken from other similar and existing sound sources, unless you know what you're doing. Large objects are given a longer initial distance before dropping any attenuation to take into account their size, as the sound sources originate from single points. This simulates a larger source object.

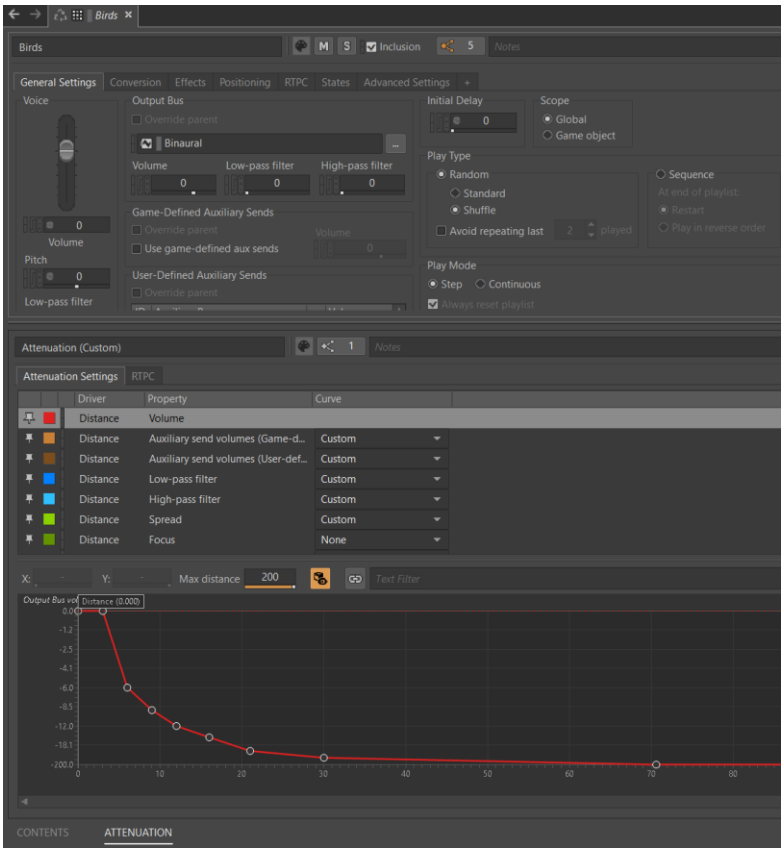


Figure 7 - The Wwise audio source settings for a Randomized Container.

Under the Positioning tab, make sure to enable 3D spatialization – both position and orientation. Additionally, enable Attenuation; otherwise sounds will simply playback the same, regardless of their position, in respect to the user (figure 8).

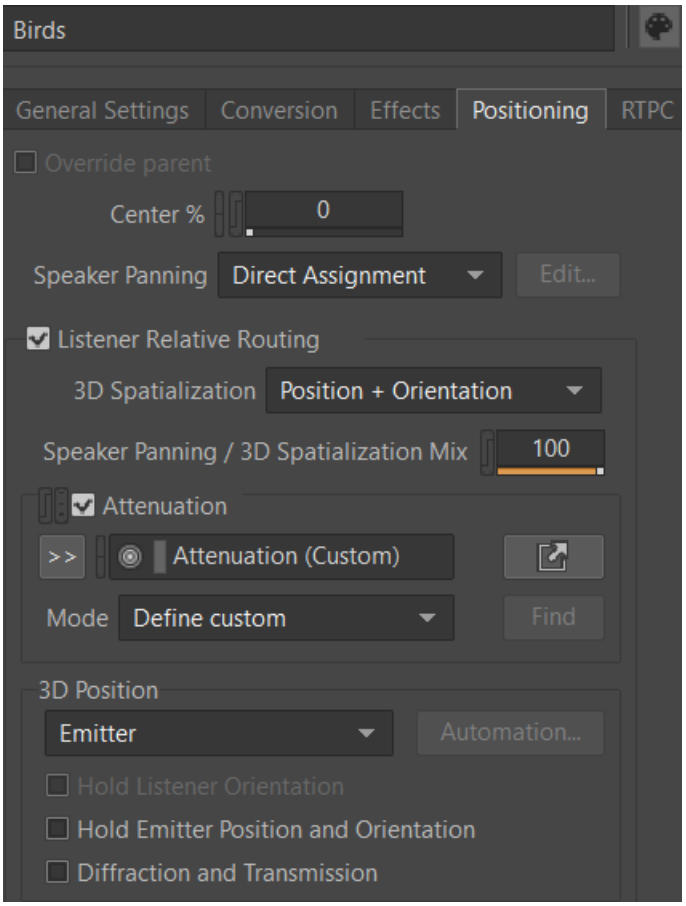


Figure 8 – Enable and set 3D spatialization and attenuation.

All audio source sound levels should be balanced between one another, relatively speaking, to create a realistic soundscape. Further documentation for sound level balancing is pending, but should be done in a software like Reaper and applied to the audio file itself. If further minor adjustments are required after processing sound levels in Reaper, the sound levels can be further modified here. E.g. modifying by 3-6dB to better balance the soundscape.

Whether working with individual SFX objects or random container, all of these settings must be set. An SFX will inherit many properties of the Random Container, but some must be set individually. For example, an ***SFX MUST be set to loop infinitely.*** Otherwise it will stop playing at the end of the clip and will not begin again until City Ditty turns it back on (e.g., when unmuting, loading a scene, or at the change of the hour when it is forcibly checked again). This must be set individually on each SFX, regardless of whether they are in a Random Container or not (figure 9). Other *Voice settings* like

Volume, pitch, etc. can be modified at both the container level, as well as individual level. Be careful of when adjusting values.

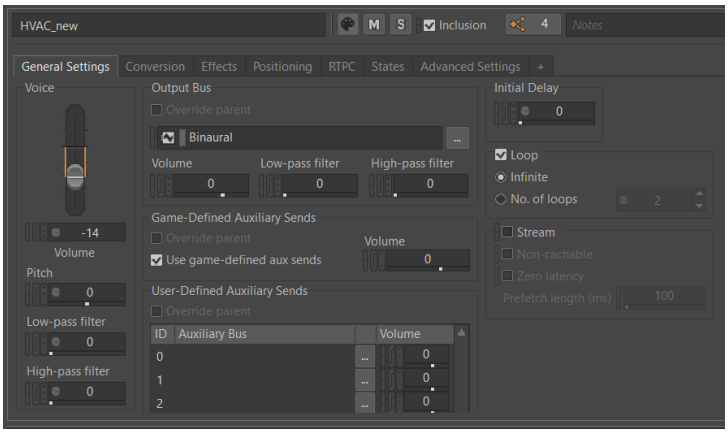


Figure 9 - Individual Audio sources have more options than the Random Container. They inherit settings from their Random Container, but there are more parameters to set.

Wwise Events Hierarchy Structure

The previous sections outline how to setup your sounds. However, a final step remains to create events that can be called from Unity/City Ditty to control how and when these audio sources are used. It requires a hierarchy in the Events tab that's near-identical to what was created in the Audio Tab. First, all new content goes into the *WorkUnityNewContributions* Work Unit to help with integrations into the main project (Figure 10).

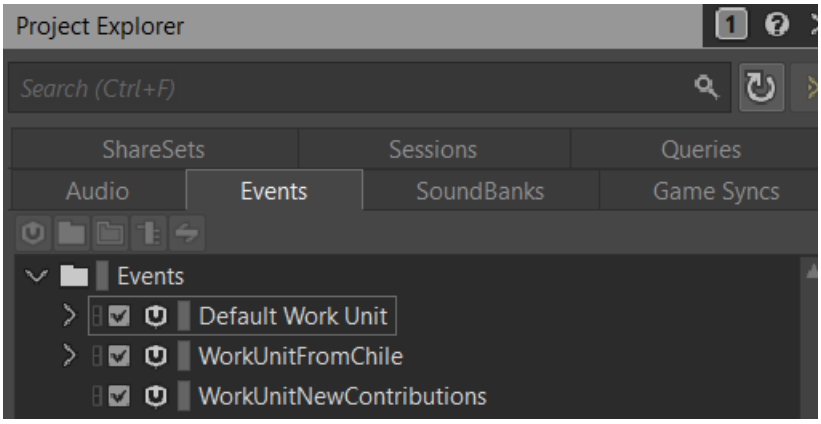


Figure 10 - Events hierarchy structure.

Second, the same thematic folder arrangements should be used in the Events tab, mirroring the Audio tab (Figure 11).

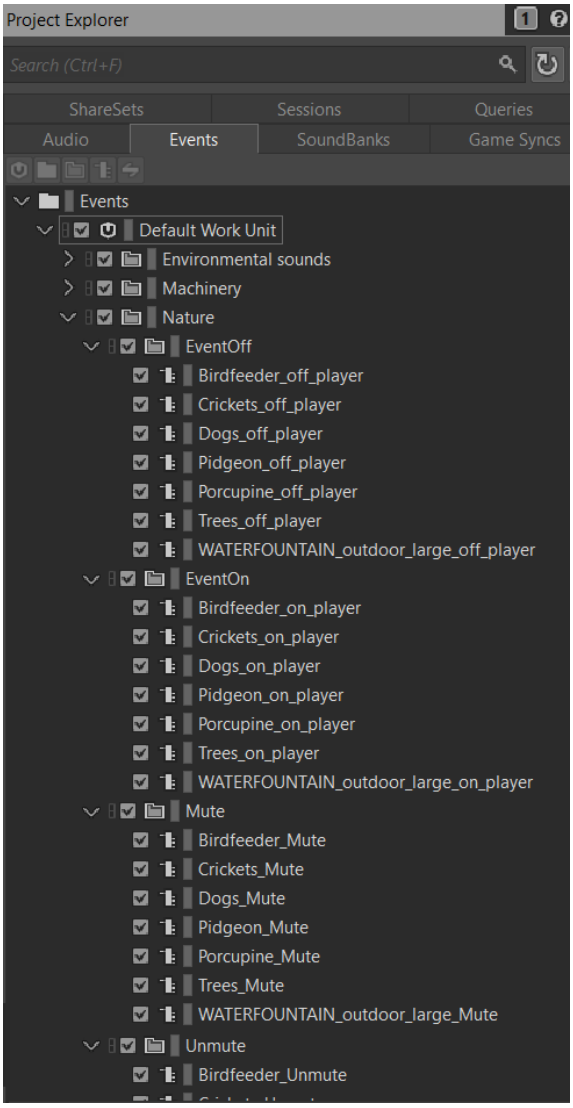


Figure 11 – Event hierarchy with thematic organization

Each Random Container or individual SFX object requires four events which should be filtered into four subfolders. One for each EventOn, EventOff, Mute, and UnMute. These are used to turn the audio sources on or off in City Ditty, or mute/unmute them, respectively. This thematic organization will help with the integration process inside Unity.

Creating the events are simple but require careful attention to detail. Right click on the Event folder and create a new event. It should be given a descriptive name that can be identified in Unity, assigned an Event Type, then given a link to the correct Audio Random Container or SFX object. Naming conventions should be consistent.

Figure 12 gives an example of a Stop event for Birds. Under Type, you can choose between many events, although at this point only Start, Stop, Mute, and Unmute are supported. Under Target, the Birds Random Container is selected. This process must be repeated for each Sound Source used in City Ditty, with all four events created separately. Each user-interactable sound source needs all four to operate properly. These events can also be shared over multiple user-interactable sound

sources. E.g., you can have 17 different user-interactable sound sources of people talking, but only need one set of 4 events to operate them all. That said, if you are using voices, you may wish to create separate event groups for *all male speakers*, *all female speakers*, and *mixed-gender speakers* to avoid audio-visual disconnects.

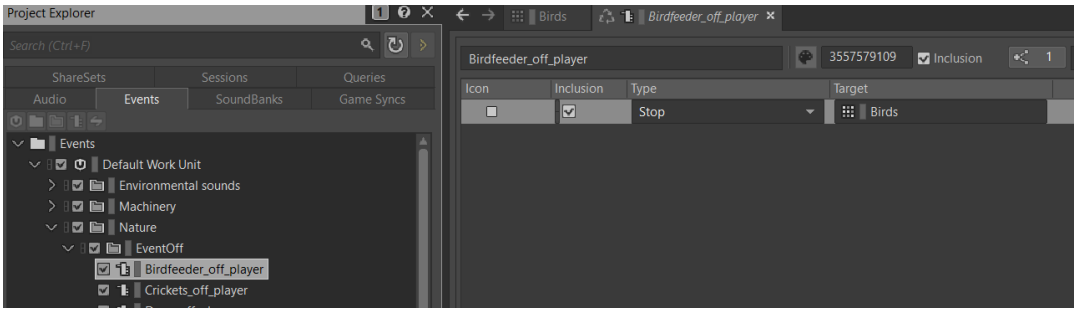


Figure 12 - Each sound source needs 4 events; each with a descriptive name, the event type, and the target SFX/Random Container selected.

In addition to the user-interactable sound sources, don't forget to create events for the roomtone. You can have one for daytime and another for nighttime, resulting in 4 or 8 additional events.

Changing existing SFX or Random Containers

Adding any new content is quicker, as I can copy/paste the Work Unit files themselves into my project. I can then organize them in a way that does not break anything. However, changing existing settings is much more tedious, as I currently need to do it manually. You will need to keep track of exactly what changes you made and let me know what to change.

Frequently Asked Questions (FAQ)

Q: Can I just add a 3D object, sound, or map by myself, without going through you?

> **A:** No. This would require the development of an incredibly complex interface that would allow you to do this. I'm a single researcher with many other responsibilities – not a full-time tech team. Please just contact me and use the guide I provided here.

Q: Can you add [function or ability]?

> **A:** Maybe. Please contact me. I'm slowly adding in features as I'm able to, however, these features typically revolve around what's necessary for the current or upcoming research project(s). There is a better chance of prioritizing this if it helps with a research project. I have a seemingly bottomless backlog of would-be-nice features and functions already. Scope, time and money are always factors.

Q: Can I help you add [function or ability]?

> **A:** Again, maybe. But please understand that even if you say that you can do it all, it will still typically require a lot of time and work from me. Currently, the best way to contribute is by 1) adding to the growing database of 3D environments, 3D objects, and sound recordings; or 2) using it as intended! For design projects, experiments, educational purposes, etc.

Q: Can I get sound level measurements from this?

A: Currently, no, and only with limited accuracy in the future. City Ditty is intended to be an accessible simulator for soundscape design – not sound level measurements. If you need accurate sound level measurements, you should use other existing acoustic software or do on site sound level measurements. In the future I do hope to be able to provide on-screen estimates that are relative to the virtual rendering, but these will never be as accurate as those.

Q: Can you make this open source?

A: Realistically, no. Although I have programmed 100% of the interfaces and interactional methods myself (aside from some basic built-in Unity functions), the creation of such a realistic, interactive world has required the purchase of many 3D models, animations, weather systems, etc. *These are not mine to give away, and many of them are not easily separable from what I have done*. City Ditty would not exist without them. The best I can do is collaborate on projects and slowly increase the database of the existing resources with the help of others.

Q: Why don't you support the Universal or High-Definition Rendering Pipelines (URP/HDRP)?

A: I started development of City Ditty in the early stages of my PhD and had fewer options available at the time. As such, I'm currently locked into several important assets that cannot easily be changed. As the focus of this software revolves around sound, enhancing the visual fidelity is a low priority. Equal support for both desktop and HMD VR is also important, and locking others into HDRP, for example, is much more resource and knowledge intensive for us all to take advantage of. In short, changing it would open a whole case of worms, requiring a ton of effort, including changing a lot of existing work by myself and others, with no real implications for research at present.